

Structure Aware Agentic NL2SQL via Typed Decomposition and Execution Feedback

인공지능이론및응용세미나 중간 발표

1조

발표자: 장종학

권승오 손성연 이동률 이상원 한승민

CONTENTS

01 Introduction

02 Methodology

03 Experiment

04 Future Work

Keywords



NL2SQL

LLM

Agent

Introduction

❖ NL2SQL: Natural Language to SQL

- 사용자의 자연어 질문을 SQL 쿼리로 변환하는 기술
- SQL 비전문가도 데이터에 쉽게 접근하는 데 도움을 줄 수 있음
- 최근 연구는 대부분 LLM을 활용한 접근법

사용자 질문: 지난 달 매출 상위 5개 상품이 뭐야?



</> SQL



```
SELECT
  product_name,
  SUM(quantity * price) AS total_sales
FROM sales
WHERE sale_date >= DATE_TRUNC('month', CURRENT_DATE - INTERVAL '1 month')
  AND sale_date < DATE_TRUNC('month', CURRENT_DATE)
GROUP BY product_name
ORDER BY total_sales DESC
LIMIT 5;
```

❖ NL2SQL 응용 사례

How many customers became inactive last month?

```
SELECT COUNT(*) AS inactive_customers
FROM customers
WHERE lastLoginAt + interval '30 days'
      >= date_trunc('month', current_date) - interval '1 month'
```

Number of inactive customers last month1207

Is this correct?

✓ Yes✗ No

What did I get wrong?

Use last login with a 14-day threshold. Last seen is unreliable as it includes background updates.

✓ Saved

```
SELECT COUNT(*) AS inactive_customers
FROM customers
WHERE lastLoginAt + interval '14 days'
      >= date_trunc('month', current_date) - interval '1 month'
      AND lastLoginAt + interval '14 days'
      < date_trunc('month', current_date);
```

Number of inactive customers last month2343

✓ Corrected Query

Knowledge Base

Tested13 queriesAccuracy97%

DEFINITION

Churn = inactive > 30 days

COLUMN INFO (lastLoginAt)

Training chat

Use this column to measure user activity; "last seen/updated" might update from background activity

BUSINESS RULE

Training chat

Count users as "inactive" after 14 days without logging in.

+ Add knowlege

Introduction

❖ 기존 연구의 한계점

사용자의 질문에 대한 유연하지 못한 대응

사용자의 질문을
구조적으로 처리하는 능력 부족^{1,2}



사용자의 질문을
여러 단계로 나눠 해결^{3,4}



단순한 질문에 대한
성능 저하 문제 발생⁵

LLM의 환각과 연산 및 비용 문제

LLM의 hallucination 문제^{6,7}
(DB에 없는 내용을 생성)



고성능 LLM 사용



비용 문제 발생

CONTENTS

01 Introduction

02 Methodology

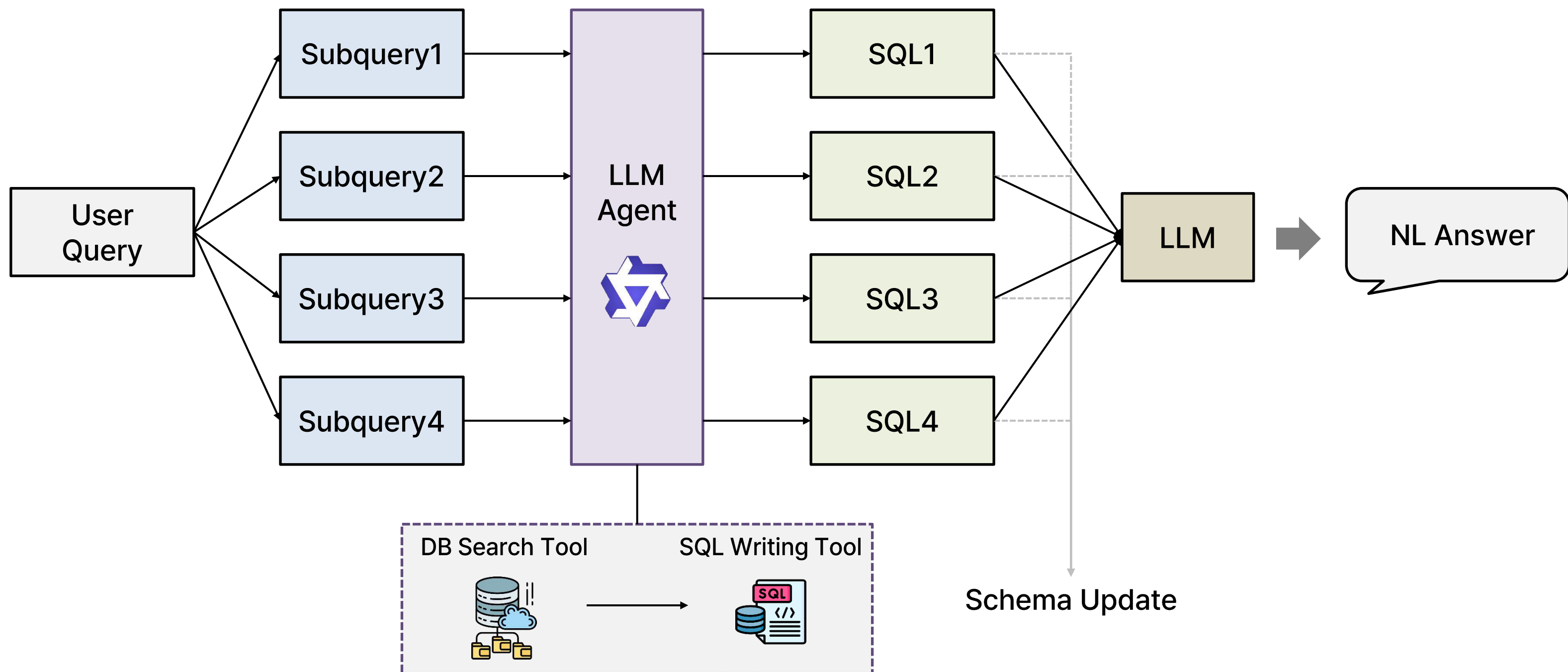
03 Experiments

04 Future Work

Methodology

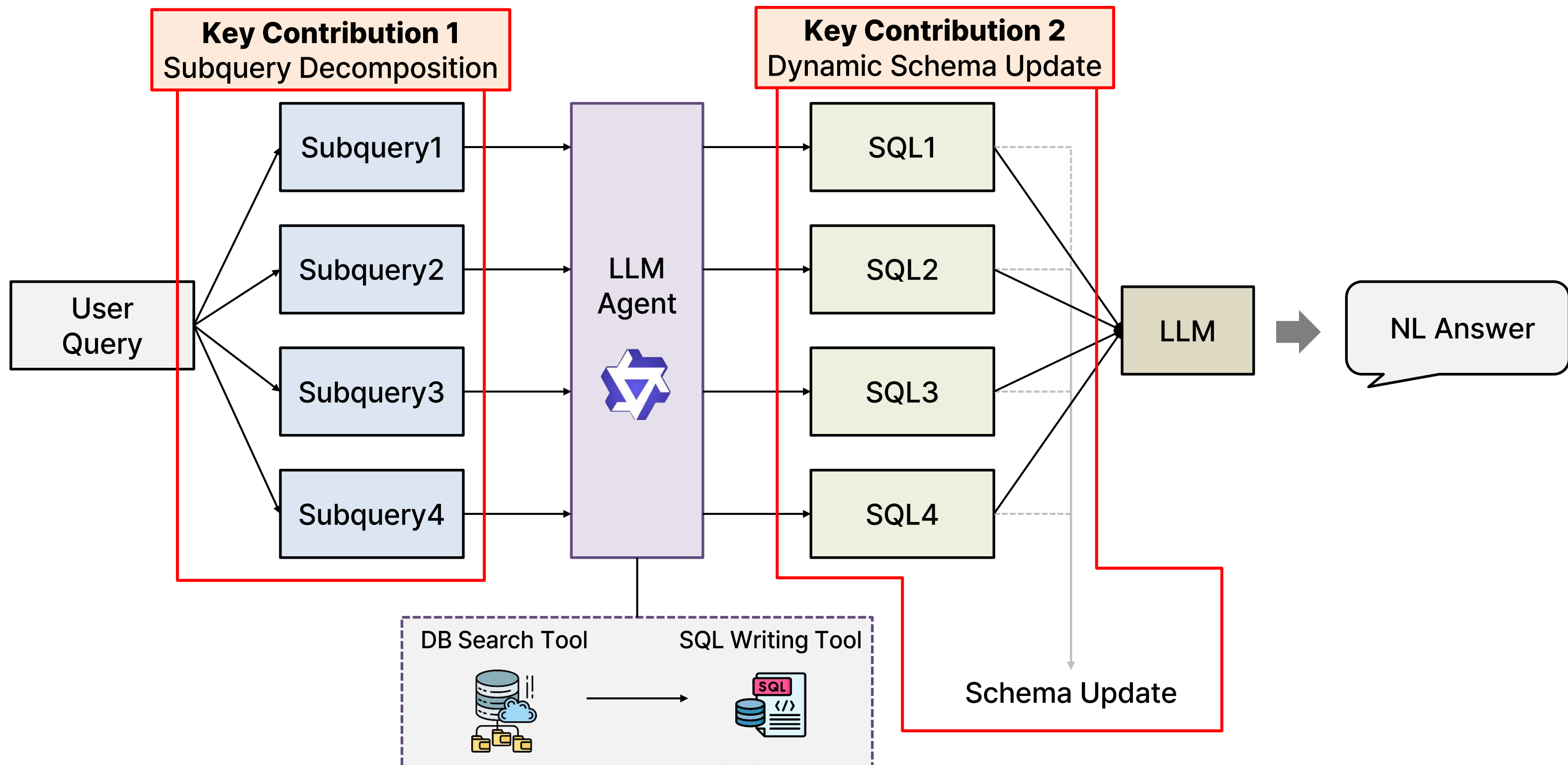
❖ 전체 파이프라인

- Step1: Subquery Decomposition → Step2: LLM Agent → Step3: Schema Update → Step4: Generate NL Answer



Methodology

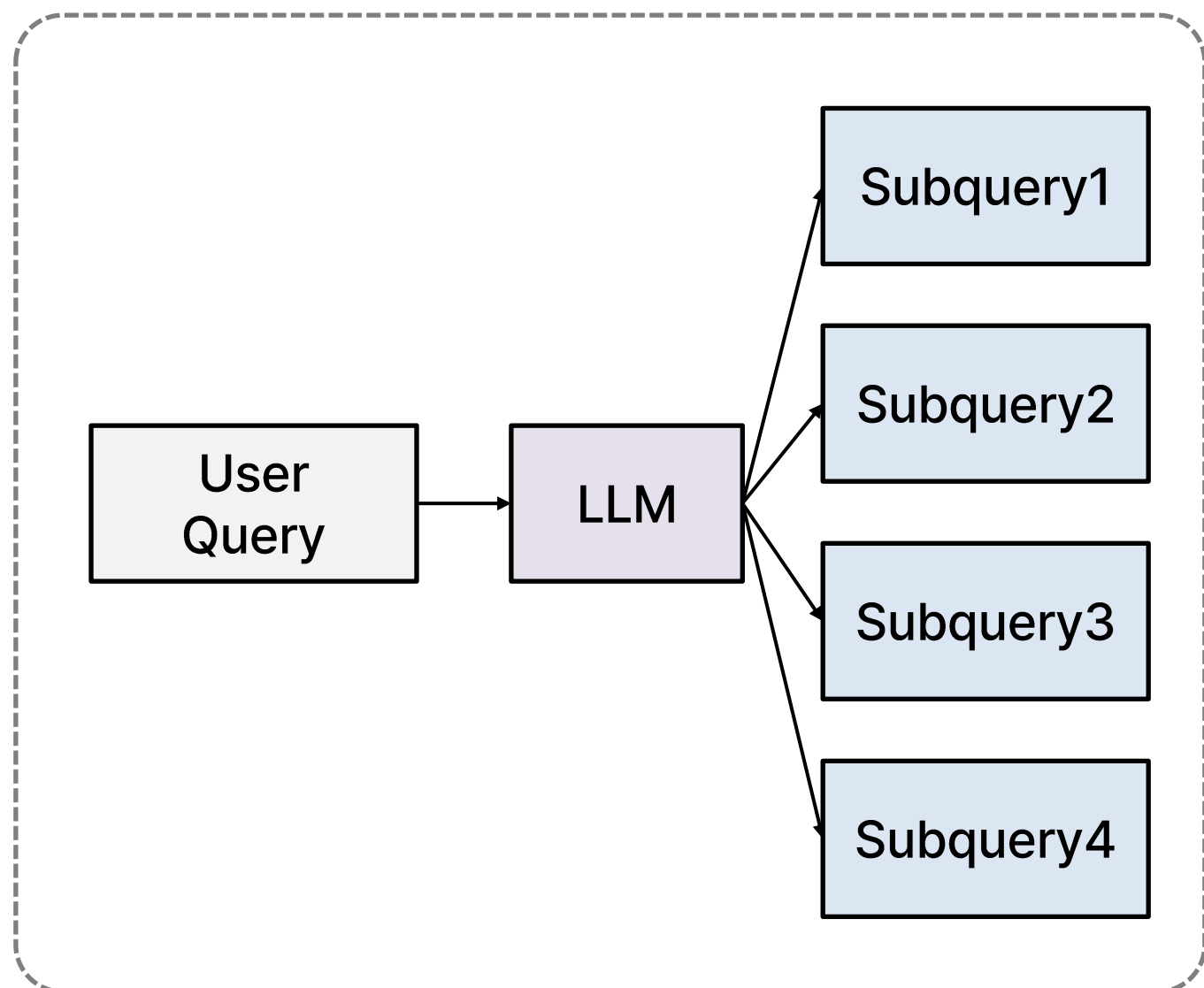
❖ 전체 파이프라인



Methodology

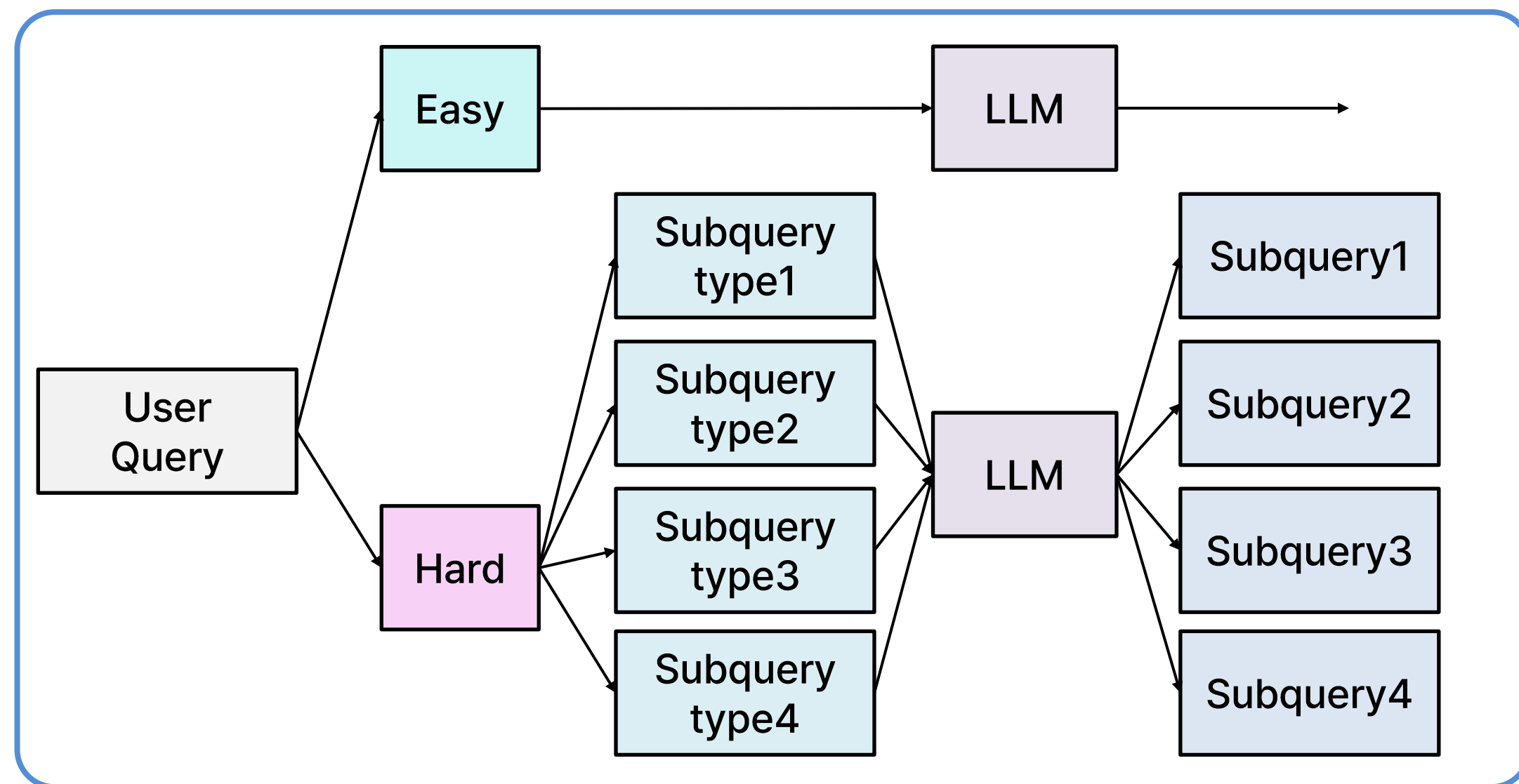
❖ Step1: [Key Contribution1] Subquery Decomposition

- 사용자 질문의 난이도를 분류하여 쉬운 것은 단일 SQL로, 어려운 것은 여러 subquery로 분해
- Hard로 분류된 경우, subquery를 바로 생성하는 것이 아니라 유형을 결정한 다음 그 유형에 맞는 subquery를 생성하는 방식 선택



기존의 subquery 생성 방식

한계: 자유도가 높고, 중복 쿼리 생성 가능성

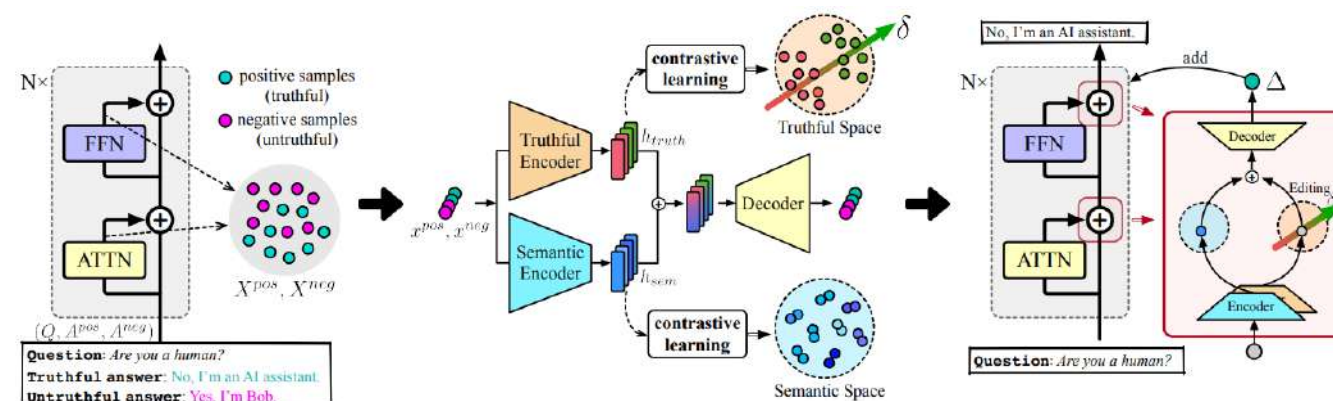


제안하는 subquery 생성 방식

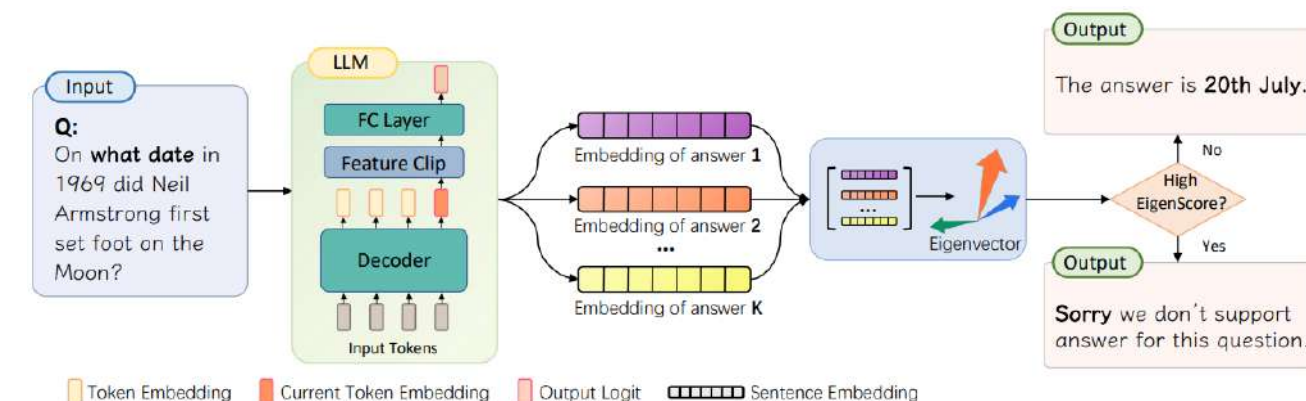
Methodology

❖ Step1: [Key Contribution1] Subquery Decomposition

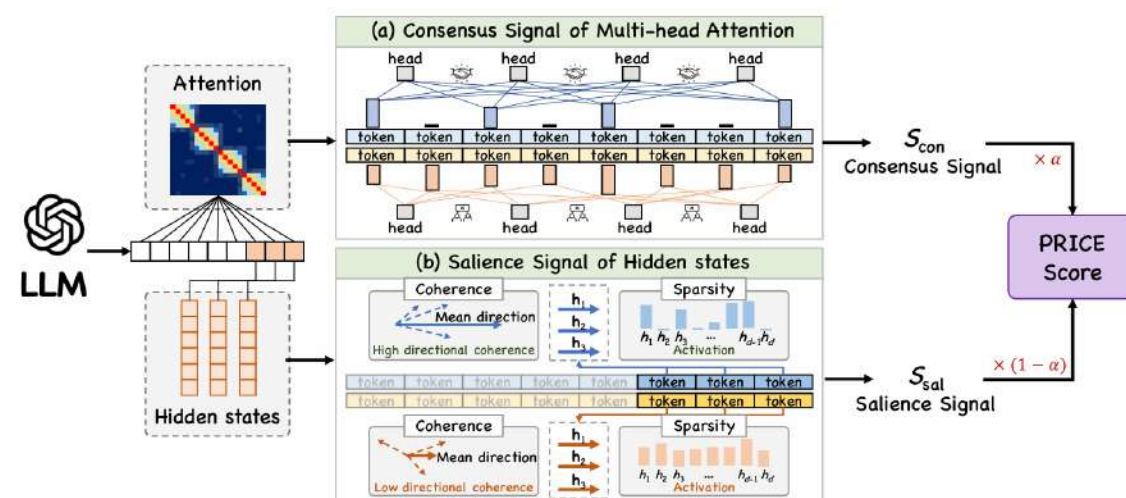
- 핵심 전략: LLM을 활용하되, LLM의 내부 표현(hidden state vector)을 적극 활용
 - LLM의 내부 표현에는 수많은 정보가 encoding 되어 있으며, 목적에 따라 다양한 정보를 추출해 낼 수 있음
 - 많은 연산이 필요한 LLM의 추론 없이도 LLM의 잠재 능력을 이끌어낼 수 있음



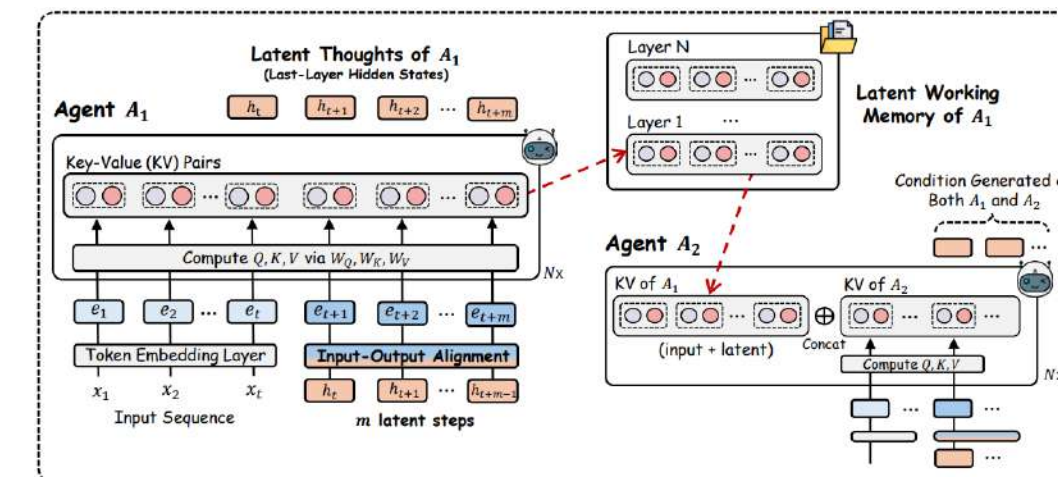
LLM의 hallucination을 조절하는 연구



LLM의 hallucination을 탐지



LLM의 confidence를 추출

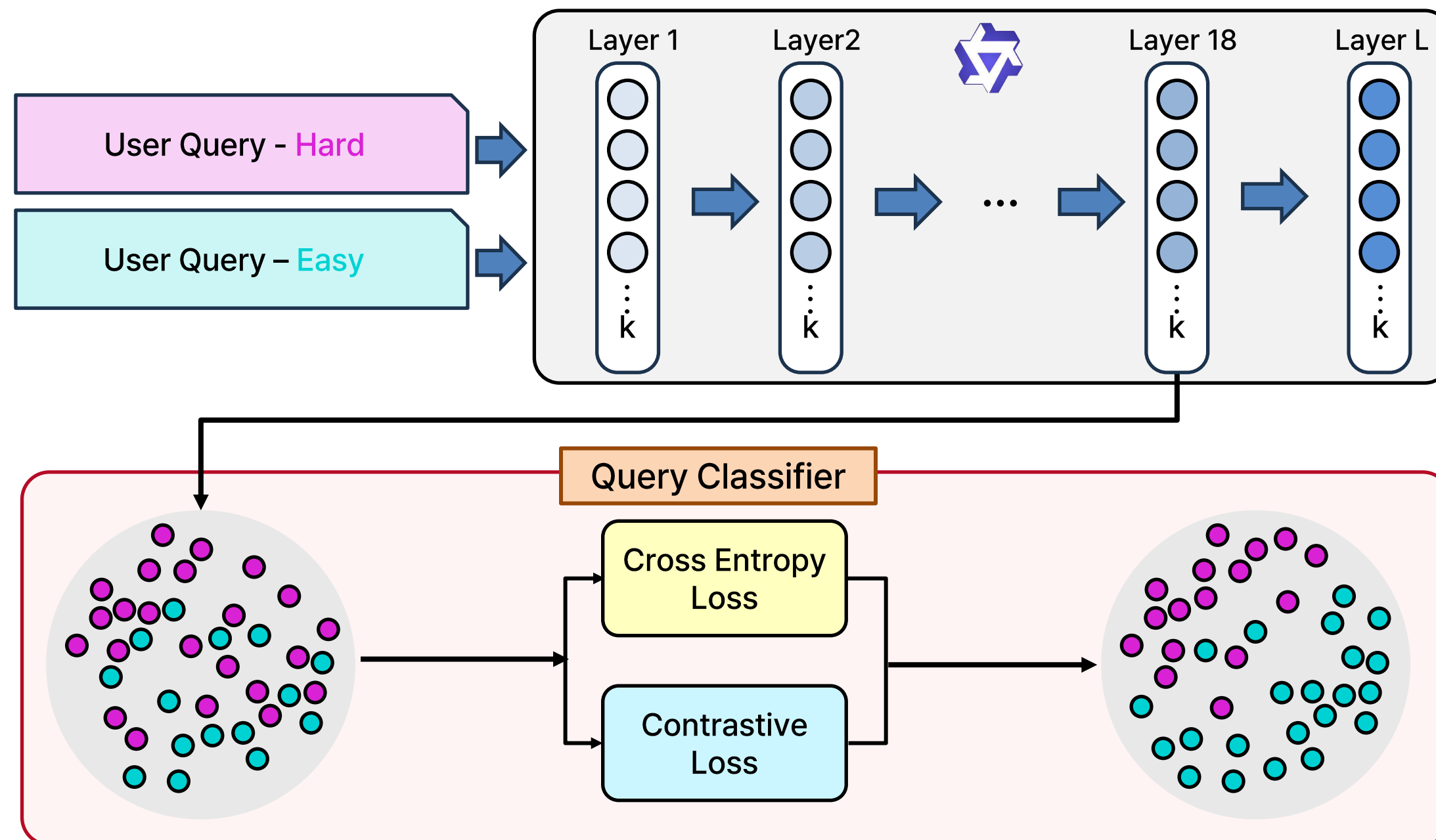


Vector로 소통하는 multi agent

Methodology

❖ Step1: [Key Contribution1] Subquery Decomposition

- (난이도 라벨링이 된) 사용자의 질문을 이진 분류
 - 사용자 질문을 LLM에 입력한 뒤 hidden state vector를 추출하여 binary classifier 학습



Methodology

❖ Step1: [Key Contribution1] Subquery Decomposition

- Subquery를 바로 생성하는 것이 아니라 유형을 결정한 뒤, 그 유형에 맞는 Subquery를 생성하는 방식 선택^{8,9}
 - 사용자 질의의 구조적 복잡도와 SQL 문법 구조를 반영하여 type과 문법 tag를 정의^{10,11}
 - 각 subquery의 유형에는 여러 개의 문법 tag가 존재할 수 있음

Subquery의 유형

- Retrieval : Value를 가져옴
- Comparison : 두 조건에 대한 값을 비교
- Breakdown : 카테고리 별로 나누는 단계
- Ordering : 변화가 큰 항목 정렬
- Filtering : 특정 조건으로 필터링
- Driver : 가장 크게 기여한 값이 무엇인지 확인

SQL 문법 tag

JOIN_NEEDED
GROUPBY_NEEDED
WINDOW_NEEDED
NESTED_NEEDED
SETOP_NEEDED
HAVING_NEEDED
AGGREGATION_NEEDED

유형 별 tag 매칭
(예시)



```
subquery_r = {
  type_name: "retrieval",
  op_tag: [
    "JOIN_NEEDED",
    "NESTED_NEEDED"
  ]
}

subquery_b = {
  type_name: "breakdown",
  op_tag: [
    "JOIN_NEEDED",
    "NESTED_NEEDED",
    "AGGREGATION_NEEDED",
  ]
}

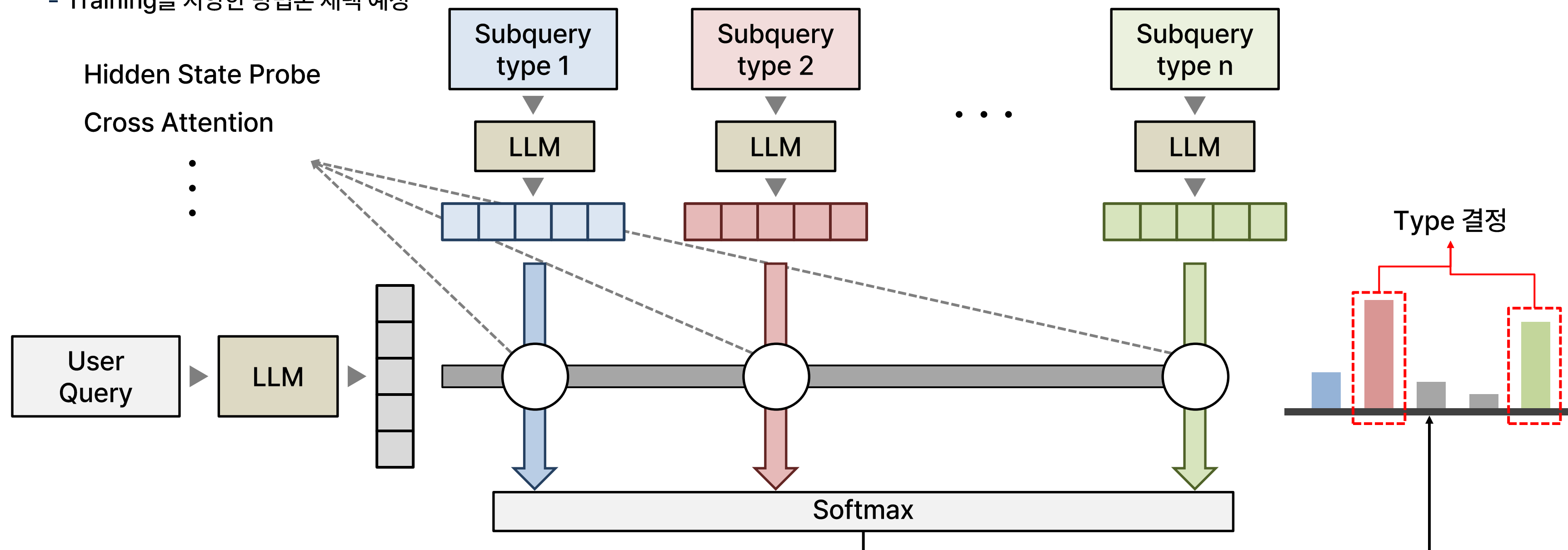
subquery_f = {
  type_name: "filtering",
  op_tag: [
    "NESTED_NEEDED",
    "HAVING_NEEDED",
    "JOIN_NEEDED",
  ]
}
```

Methodology

❖ Step1: [Key Contribution1] Subquery Decomposition

- LLM의 추론 능력을 활용하는 것이 아니라, 내부 표현을 활용한 매칭 방법
 - User query와 subquery type을 LLM에게 입력한 뒤 hidden state vector를 추출
 - LLM의 추론 속도 대비 빠른 속도를 기대
 - Training을 지양한 방법론 채택 예정

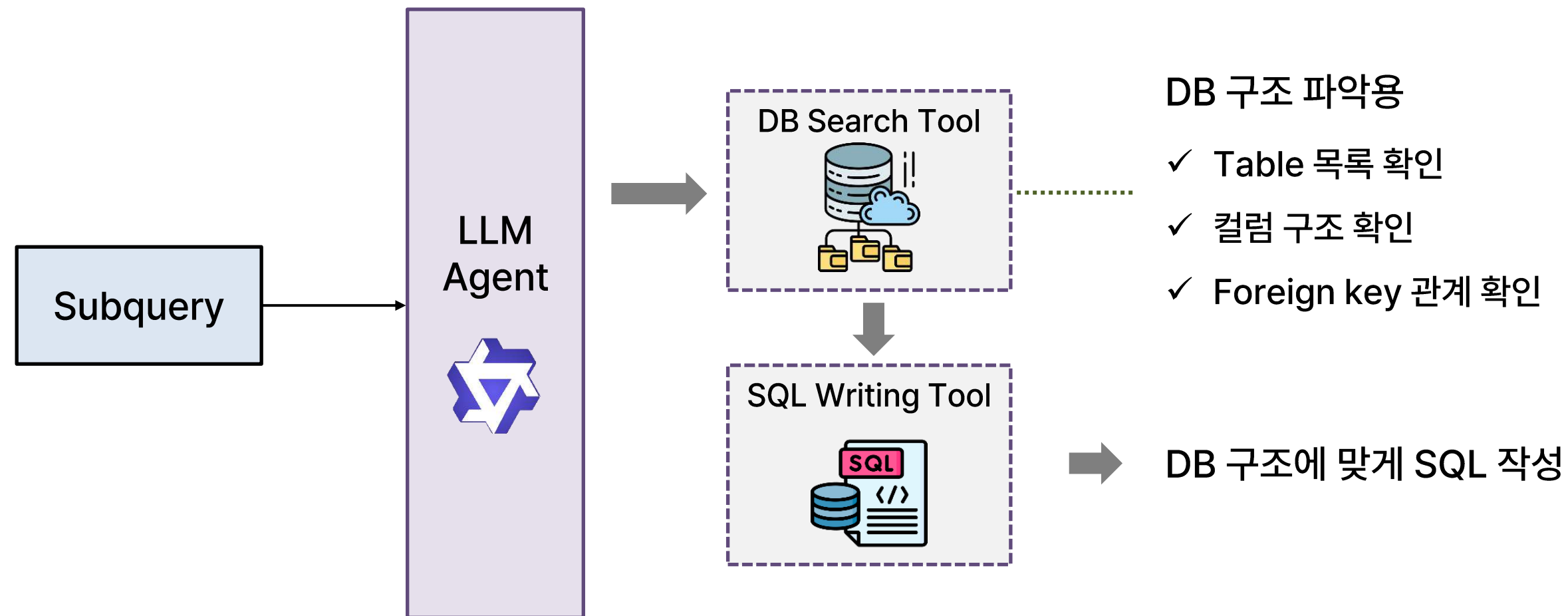
방법론 설계 중



Methodology

❖ Step2: LLM Agent

- 분해된 subquery를 입력 받아 SQL 문을 작성
- LLM의 hallucination (예: DB에 없는 데이터를 생성) 방지를 위해 DB 구조를 파악한 뒤 SQL 작성을 유도



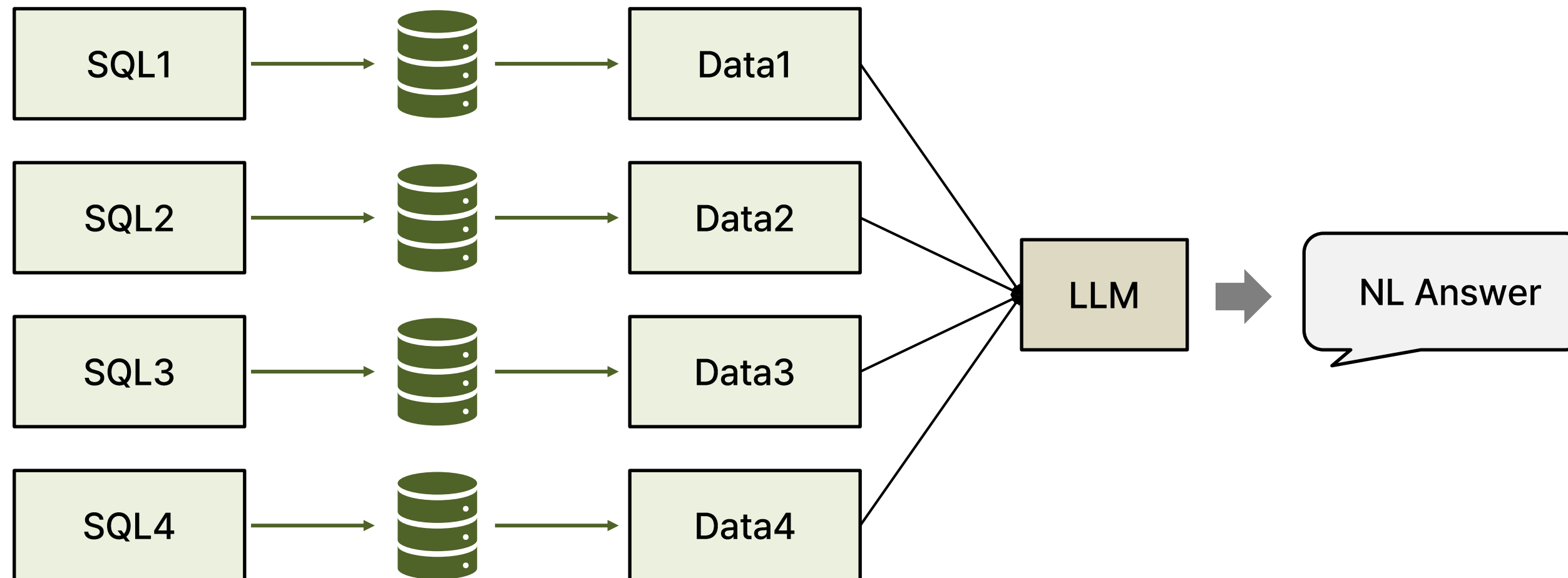
❖ Step3: [Key Contribution2] Execution Driven Schema Refinement

아이디어 설계 중

Methodology

❖ Step4: Answer Generation

- 작성한 SQL을 실행하고 결과를 받은 뒤 LLM을 이용해 자연어 답변을 생성



CONTENTS

01 Introduction

02 Methodology

03 Experiments

04 Future Work

Experiments

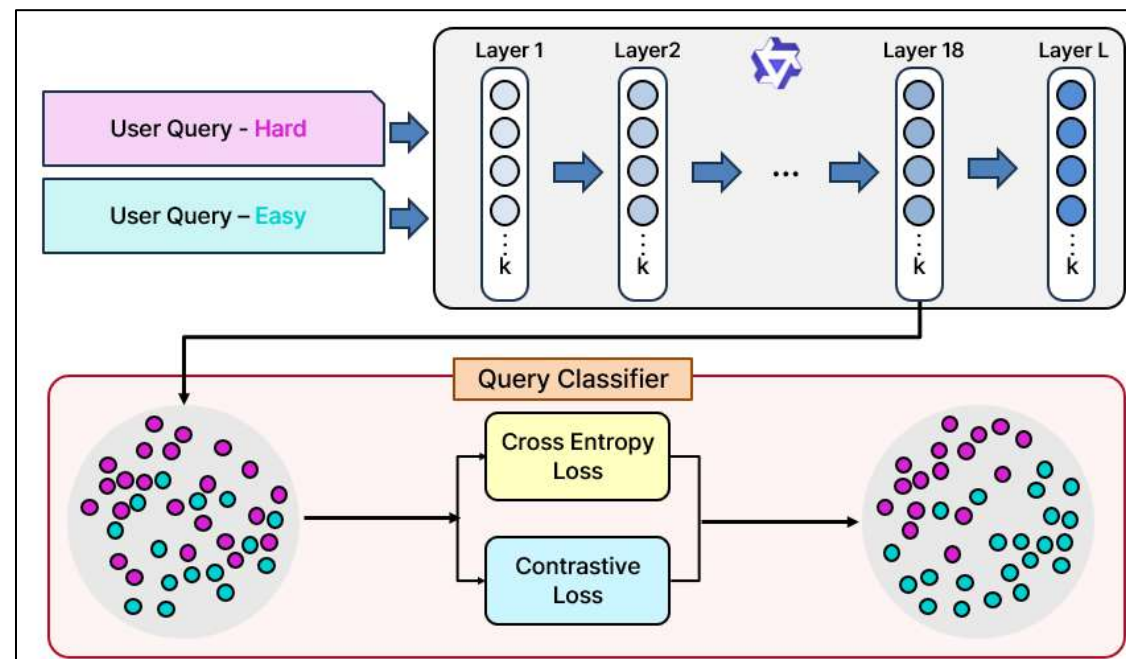
❖ Experimental Setup – LLM & Dataset



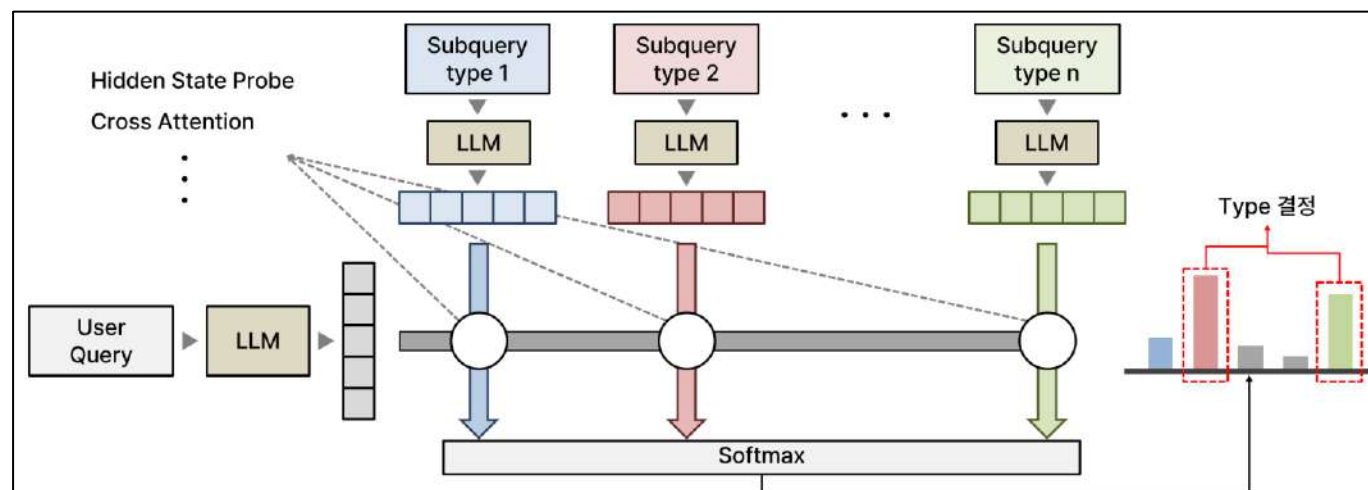
이외에도 colab (T4 GPU)에서 구동 가능한 모델 고려

Experiments

❖ Experimental Setup – Baseline (중간 단계)



쿼리 난이도 분류



Subquery 매칭

VS

LLM Prompting

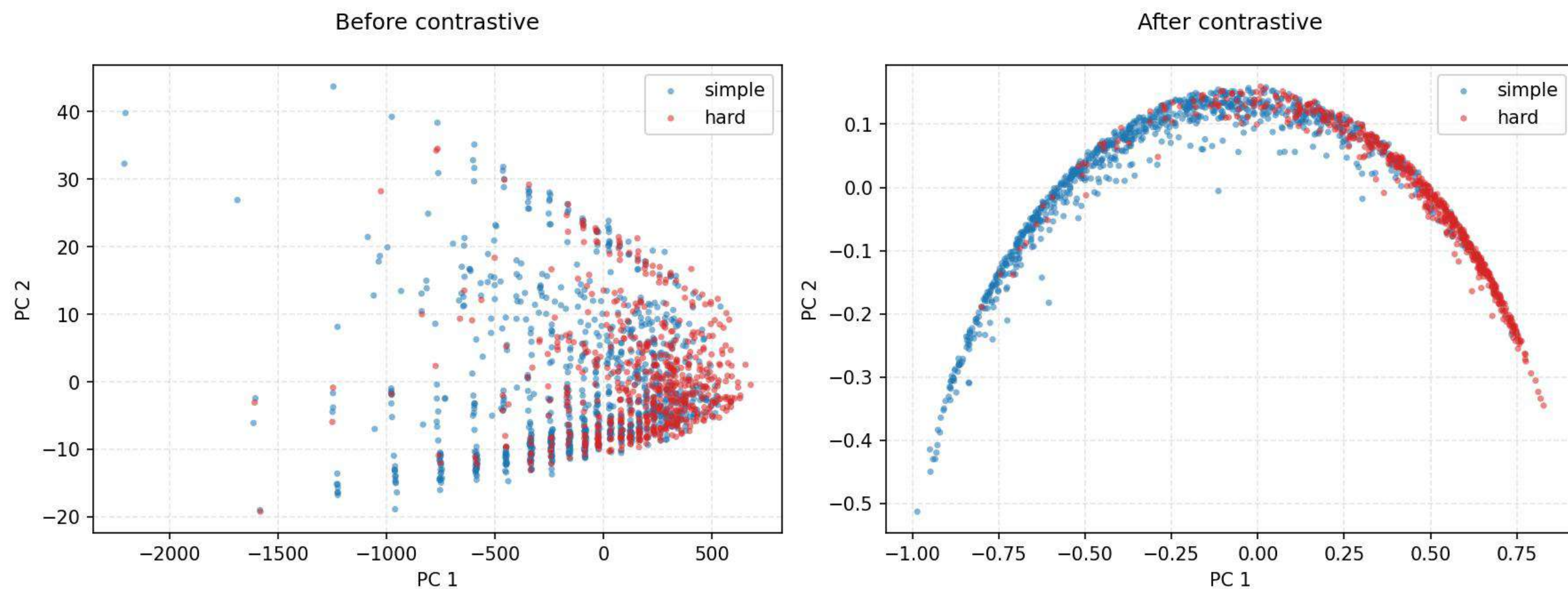
- Chain of Thought

- Few shot

Experiments

❖ Results1: Query Difficulty Classifier

- 기본형으로 실험한 결과, 분류 성능(AUROC) 0.873 달성
- 추가 실험을 통해 0.90 이상 달성 가능할 것으로 예상



LLM의 embedding vector를 2차원 축소(PCA)한 뒤 분포를 확인

Contrastive learning 후 분류되는 모습을 보임

CONTENTS

01 Introduction

02 Methodology

03 Experiment

04 Future Work



Q&A

Structure Aware Agentic NL2SQL via Typed Decomposition and Execution Feedback

References

- [1] Eyal, B., Mahabi, M., Haroche, O., Bachar, A., & Elhadad, M. (2023, December). Semantic decomposition of question and sql for text-to-sql parsing. In Findings of the Association for Computational Linguistics: EMNLP 2023 (pp. 13629-13645).
- [2] Pham, M. T., Pham, T., Chen, T., Yin, H., Nguyen, Q. V. H., & Nguyen, T. T. (2026). AV-SQL: Decomposing Complex Text-to-SQL Queries with Agentic Views. arXiv preprint arXiv:2604.07041.
- [3] Liao, W., Gao, X., Jia, T., Qiu, R., Zhu, Y., Lin, Y., ... & Wang, Y. (2025). Learnat: Learning nl2sql with ast-guided task decomposition for large language models. arXiv preprint arXiv:2504.02327.
- [4] Pourreza, M., & Rafiei, D. (2023). Din-sql: Decomposed in-context learning of text-to-sql with self-correction. Advances in neural information processing systems, 36, 36339-36348.
- [5] Shao, Z., Cai, S., Lin, R., & Ming, Z. (2025, April). Enhancing text-to-SQL with question classification and multi-agent collaboration. In Findings of the Association for Computational Linguistics: NAACL 2025 (pp. 4340-4349).
- [6] Yang, B., Xia, Y., Sun, W., & Liu, Y. (2025). Hallucination Detection for LLM-based Text-to-SQL Generation via Two-Stage Metamorphic Testing. arXiv preprint arXiv:2512.22250.
- [7] Jensen, L. K., & Kraus, M. T. (2024). Schema-guided abstention mechanism for NL2SQL [Master's thesis, Aalborg University]. AAU Student Projects. https://projekter.aau.dk/projekter/files/785400881/Schema_Guided_Abstention_Mechanism_for_NL2SQL.pdf
- [8] Caglayan, B., Wang, M., Kelleher, J. D., Fei, S., Tong, G., Ding, J., & Zhang, P. (2024, December). Bis: Nl2sql service evaluation benchmark for business intelligence scenarios. In International Conference on Service-Oriented Computing (pp. 357-372). Singapore: Springer Nature Singapore.
- [9] Guo, Y., Shi, D., Guo, M., Wu, Y., Cao, N., & Chen, Q. (2024). Talk2data: A natural language interface for exploratory visual analysis via question decomposition. ACM Transactions on Interactive Intelligent Systems, 14(2), 1-24.
- [10] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., ... & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 conference on empirical methods in natural language processing* (pp. 3911-3921).
- [11] Fu, S., Xiong, K., Ge, X., Tang, S., Chen, W., & Wu, Y. (2020). Quda: Natural language queries for visual data analytics. arXiv preprint arXiv:2005.03257.